

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY



(SUBJECT: ADVANCED MACHINE LEARNING)
(SUBJECT CODE:24SAL051)

AN INDIVIDUAL TASK REPORT ON
“Comparative Analysis of SVM, Naive Bayes and
Decision Tree for Heart Disease Prediction”

Submitted in the partial fulfilment of the requirements for the Fifth semester

BACHELOR OF TECHNOLOGY
IN
Artificial Intelligence & Machine Learning

Submitted By,
Pavan S Poojary
01SU24AI071

UNDER THE GUIDANCE OF
Prof. Mahesh Kumar V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146

2026-27

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY

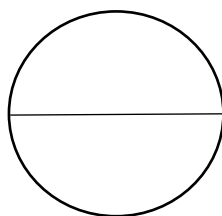


Department of Artificial Intelligence & Machine Learning

CERTIFICATE

This is to certify that, **Pavan S Poojary (01SU24AI071)**, has satisfactorily completed the assessment (Individual Task Report) in **Advanced Machine Learning (24SAL051)** prescribed by the Srinivas University for the 5th semester B. Tech. course during the year **2026-27**.

MARKS AWARDED



Staff In charge

Prof. Mahesh Kumar V B

TABLE OF CONTENTS

Sl.no	Description	Page no.
1	Abstract	1
2	Introduction	2
3	Dataset and Data Understanding	4
4	Data Preprocessing	6
5	Machine Learning Methodology	9
6	Implementation	12
7	Model Evaluation	34
8	Results and Analysis	40
9	Prediction and Discussion	44
10	Conclusion	46
11	References	47

Abstract

Heart disease is one of the major health challenges, and its early prediction can help in providing timely medical attention and improving healthcare decision-making. This project presents a machine learning-based approach for predicting heart disease using three classification algorithms: **Support Vector Machine (SVM), Naive Bayes, and Decision Tree**. A heart disease dataset containing relevant clinical and patient-related attributes is used to develop and evaluate the predictive models.

The dataset is first subjected to appropriate **data preprocessing**, including data inspection, feature selection, encoding of categorical attributes, feature scaling, and division into training and testing datasets. The selected machine learning algorithms are then trained using the processed data. Their performance is evaluated using **accuracy, precision, recall, F1-score, ROC-AUC, classification reports, and confusion matrices**. The SVM model obtained approximately **88.52% accuracy, 83.87% precision, 92.86% recall, 88.14% F1-score, and 96.43% ROC-AUC** in the implemented analysis.

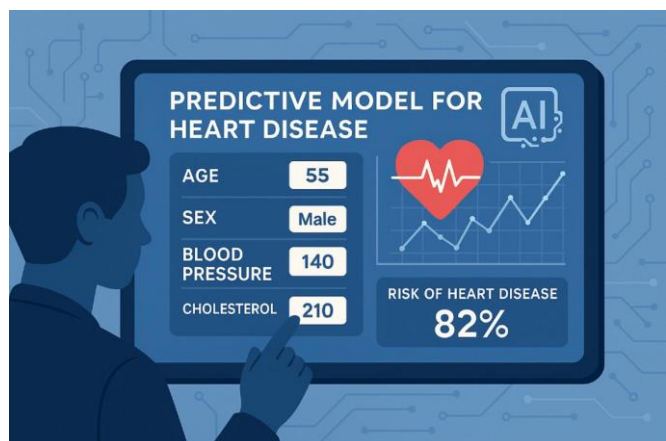
The project also demonstrates the prediction of heart disease for new patient data using the trained models. A comparative analysis of SVM, Naive Bayes, and Decision Tree helps identify differences in their classification performance and effectiveness. The results demonstrate that machine learning can be effectively applied to heart disease prediction and can provide useful computational support for analysing healthcare data. However, such predictions are intended for academic and decision-support purposes and should not replace professional medical diagnosis.

1. Introduction

Heart disease is one of the major health challenges affecting people worldwide. It refers to a group of conditions that affect the heart and blood vessels and can significantly impact an individual's quality of life. Common factors associated with heart disease include high blood pressure, high cholesterol, diabetes, obesity, smoking, physical inactivity, unhealthy dietary habits, age, and family history. Since several of these factors can occur together, analysing multiple patient characteristics can help identify patterns associated with heart disease.

Early prediction of heart disease is important because timely identification of potential risk can support further medical evaluation and appropriate treatment. Traditional diagnosis generally involves medical history, physical examination, laboratory tests, and other clinical procedures. With the increasing availability of healthcare data, **Machine Learning (ML)** provides an additional computational method for analysing patient information. Machine learning algorithms can learn patterns from existing data and use these patterns to make predictions for new observations.

In a machine learning-based heart disease prediction system, different patient attributes are provided as input to a trained classification model. The model analyses the relationship between these attributes and the target outcome and predicts whether the patient belongs to the heart disease or no-heart-disease category. Such systems can assist in analysing large amounts of structured healthcare data and can provide useful computational support for medical research and decision-making.



The development of an effective prediction model requires proper **data preprocessing**. Healthcare datasets may contain numerical and categorical attributes, different feature scales, missing or inconsistent values, and irrelevant information. Therefore, the dataset must first be inspected and prepared before applying machine learning algorithms. In this project, the data is processed through feature preparation, selection of required attributes, categorical encoding, feature scaling where necessary, and division into training and testing datasets. These steps help ensure that the machine learning algorithms receive suitable input data.

This project focuses on the application and comparison of three supervised machine learning classification algorithms: **Support Vector Machine (SVM)**, **Naive Bayes**, and **Decision Tree**. These algorithms use different mathematical and computational approaches to classify patient records. SVM attempts to determine an effective boundary between different classes. Naive Bayes uses probability-based classification, while Decision Tree creates a tree-like structure of conditions to reach a classification decision. Applying all three algorithms to the same dataset allows their performance to be analysed under similar conditions.

The project follows a systematic machine learning workflow beginning with dataset loading and understanding. The dataset is examined to identify its structure, attributes, and target variable. Exploratory analysis is performed to understand the distribution of the data and identify useful patterns. The required features are then prepared for machine learning by applying appropriate preprocessing techniques. After preprocessing, the dataset is divided into training and testing portions. The training data is used to build the models, while the testing data is used to evaluate their ability to classify previously unseen records.

Model evaluation is an important part of this project. Instead of considering only one performance measure, multiple evaluation metrics are used to obtain a more complete understanding of each algorithm. These include **Accuracy, Precision, Recall, F1-Score, and ROC-AUC**. Confusion matrices and classification reports are also generated to analyse the number of correctly and incorrectly classified samples. These measures allow the performance of SVM, Naive Bayes, and Decision Tree to be compared in a systematic manner.

In the implemented analysis, the **SVM model achieved an accuracy of approximately 88.52%**, with a precision of 83.87%, recall of 92.86%, F1-score of 88.14%, and ROC-AUC of 96.43%. These results indicate that SVM provided strong classification performance on the test data, particularly in identifying positive heart disease cases. The results of the other algorithms are also considered to determine their relative strengths and weaknesses and to identify the most suitable model for the given dataset.

The project further demonstrates the use of the trained models for predicting the outcome of **new patient data**. By providing previously unseen patient attributes to the trained models, the system generates a predicted class. This demonstrates the practical application of the developed machine learning pipeline and shows how classification algorithms can be used to analyse new observations after being trained on historical data.

Overall, this project demonstrates an end-to-end approach to heart disease prediction using machine learning. It covers dataset understanding, preprocessing, feature preparation, model development, performance evaluation, comparison, and prediction on new data. The comparison of SVM, Naive Bayes, and Decision Tree provides a clear understanding of how different classification techniques perform on the same healthcare dataset. The project also highlights the importance of selecting suitable features, preprocessing the data correctly, and using multiple evaluation metrics when developing predictive machine learning systems for healthcare applications. The developed models are intended for academic and analytical purposes and should not replace professional medical diagnosis or clinical judgement.

2. Dataset and Data Understanding

2.1 Heart Disease Dataset

The dataset used in this project is the **Heart Disease Cleveland Dataset**. The objective of using this dataset is to analyse patient-related clinical attributes and predict whether a patient is likely to have heart disease. The project applies three classification algorithms: **Support Vector Machine (SVM), Gaussian Naive Bayes, and Decision Tree**.

The dataset contains different patient-related clinical attributes that can be used to identify patterns associated with heart disease. These attributes include both numerical and categorical information. Before applying machine learning algorithms, the dataset is examined and prepared through appropriate preprocessing techniques.

The preprocessing process includes **checking for missing values, identifying categorical features, applying One-Hot Encoding, dividing the dataset into training and testing sets, and applying feature scaling**. These steps ensure that the data is converted into a suitable format for machine learning algorithms.

2.2 Dataset Features

The features in the dataset represent different characteristics of the patients that may be useful for predicting heart disease. Since the dataset contains different types of variables, each feature must be appropriately prepared before model training.

Numerical features contain measurable values, whereas categorical features represent different categories or groups. Categorical features are converted into numerical form using **One-Hot Encoding**. This allows the machine learning algorithms to process categorical information effectively.

Feature scaling is also applied as part of the preprocessing pipeline. Scaling is particularly useful for algorithms such as SVM because features with larger numerical ranges can otherwise have a greater influence on the model. The project therefore prepares the input data systematically before training the models.

2.3 Target Variable

The target variable represents the final output that the machine learning models need to predict. In this project, patients are classified into two categories:

- **No Disease**
- **Disease**

Therefore, the project is treated as a **binary classification problem**. The models learn patterns from the training data and use those patterns to predict the class of previously unseen patient records.

The distribution of the target classes is also analysed during exploratory data analysis. Understanding the distribution of the classes is important because an imbalance between classes can affect the interpretation of model performance.

2.4 Dataset Characteristics

The dataset contains a combination of clinical attributes that provide information about the health characteristics of patients. These attributes are used as input variables for the machine learning algorithms.

Since the dataset contains numerical and categorical features, preprocessing plays an important role in the overall machine learning workflow. Missing-value checking, categorical feature identification, One-Hot Encoding, feature scaling, and train-test splitting are performed before model training.

After preprocessing, the resulting dataset is suitable for training the three selected classification algorithms. The same prepared data is supplied to **SVM, Naive Bayes, and Decision Tree**, allowing their performance to be compared under similar conditions.

2.5 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed before model development to understand the structure and distribution of the dataset. EDA helps identify important patterns and provides a better understanding of the relationship between the available attributes and the target variable.

One important analysis is the distribution of the target classes. The number of patients belonging to the **No Disease** and **Disease** categories is examined using graphical representation. Such analysis helps understand whether the target classes are reasonably distributed and provides useful information before model training.

Overall, the dataset understanding stage provides the foundation for the machine learning process. The available patient attributes are examined, the target variable is identified, and the data is prepared for subsequent modelling. After completing this stage, the processed dataset can be used for training and evaluating **SVM, Naive Bayes, and Decision Tree** classification models.

3. Data Preprocessing

3.1 Data Loading

The first step in developing the heart disease prediction system is loading the Heart Disease Cleveland Dataset into the Python environment. The dataset is imported into a Pandas DataFrame so that its records and attributes can be easily examined and processed. Loading the data correctly is important because all subsequent preprocessing and machine learning operations depend on the quality and structure of the input dataset.

After loading the dataset, its basic structure is examined to understand the number of records, number of features, data types, and target variable. This provides an initial understanding of the information available for building the prediction models.

3.2 Data Inspection

Data inspection is performed to identify the characteristics of the dataset before applying any transformation. The first few records can be displayed to understand how the features are represented. The shape of the dataset is also checked to determine the number of observations and attributes.

The data types of the different columns are examined because the preprocessing method depends on whether a feature is numerical or categorical. Numerical variables can generally be used after appropriate scaling, while categorical variables need to be converted into a numerical representation.

Data inspection also helps identify potential problems such as missing values, unexpected data types, duplicate records, or unsuitable attributes. Performing this step before model training reduces the possibility of errors during implementation.

3.3 Handling Missing Values

Missing values can negatively affect machine learning algorithms because most classification models require complete input data. Therefore, the dataset is checked for missing or null values before proceeding with model development.

If missing values are present, an appropriate treatment method can be applied depending on the nature of the feature. Numerical values may be handled using statistical measures such as the mean or median, while categorical values can be handled using the most frequent category or another suitable method.

In this project, missing-value checking is included as part of the preprocessing workflow. This ensures that the quality of the input data is examined before the models are trained.

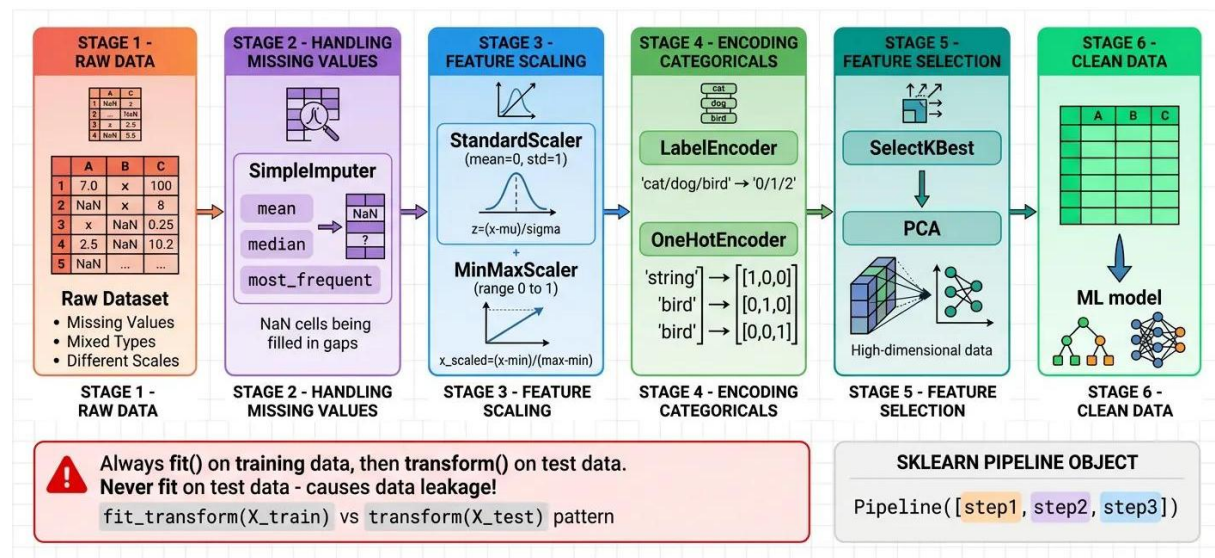
3.4 Feature Selection

Feature selection involves identifying the attributes that are useful for predicting heart disease. Not every available column necessarily contributes equally to the prediction task. Irrelevant or

unnecessary information can increase model complexity and may negatively affect prediction performance.

The required patient-related attributes are therefore separated from the target variable. The selected features are represented as the input variable **X**, while the heart disease classification is represented as the target variable **y**.

Proper feature selection provides the machine learning algorithms with meaningful information and establishes a clear separation between the input data and expected output.



3.5 Encoding Categorical Features

The dataset contains categorical features that cannot be directly processed by most machine learning algorithms in their original textual or categorical form. To convert these variables into a suitable numerical representation, **One-Hot Encoding** is applied.

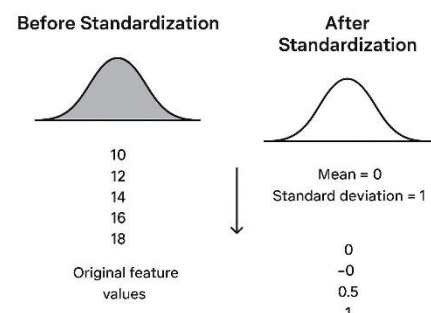
One-Hot Encoding creates separate binary columns for the different categories of a categorical feature. For example, if a feature contains three categories, the encoding process can represent these categories using separate numerical columns. This prevents the algorithm from incorrectly assuming that one category is numerically greater or smaller than another.

The project uses **OneHotEncoder** as part of the preprocessing pipeline. This ensures that categorical variables are transformed consistently for both training and testing data.

3.6 Feature Scaling

Feature scaling is another important preprocessing step used in the project. Numerical attributes can have different ranges. For example, one feature may contain values in a small range while another feature may contain considerably larger values.

When features have significantly different scales, some machine learning algorithms may give greater



importance to features with larger numerical values. This is particularly relevant to **Support Vector Machine**, which relies on distances and the position of observations in feature space.

The project uses **StandardScaler** to standardize numerical features. Standardization transforms the numerical features to a comparable scale, helping the algorithms process the input data more effectively.

3.7 Train-Test Split

After preprocessing, the dataset is divided into training and testing subsets. The **training dataset** is used to teach the machine learning algorithms the patterns present in the patient data. The **testing dataset** is kept separate and is used to determine how well the trained models perform on unseen data.

Separating the data into training and testing portions is important for evaluating the generalization capability of the models. A model should not simply memorize the training examples; it should be capable of making accurate predictions for new observations.

The same processed training and testing data are used for the three selected algorithms—**SVM, Naive Bayes, and Decision Tree**—so that their performance can be compared fairly.

3.8 Preprocessing Pipeline

The preprocessing operations are organized into a systematic workflow using Python's machine learning libraries. The project uses components such as **ColumnTransformer, OneHotEncoder, StandardScaler, and Pipeline** to manage the transformation of different feature types.

A preprocessing pipeline has several advantages. It ensures that the same transformations applied to the training data are also applied to the testing data. It also reduces the possibility of inconsistent preprocessing and makes the complete machine learning workflow easier to maintain.

The overall preprocessing process can be summarized as:

Raw Dataset → Data Inspection → Missing-Value Check → Feature Selection → Categorical Encoding → Feature Scaling → Train-Test Split → Model Training

This preprocessing stage prepares the heart disease dataset for the next stage of the project, where the **SVM, Naive Bayes, and Decision Tree** algorithms are trained and evaluated.

4. Machine Learning Methodology

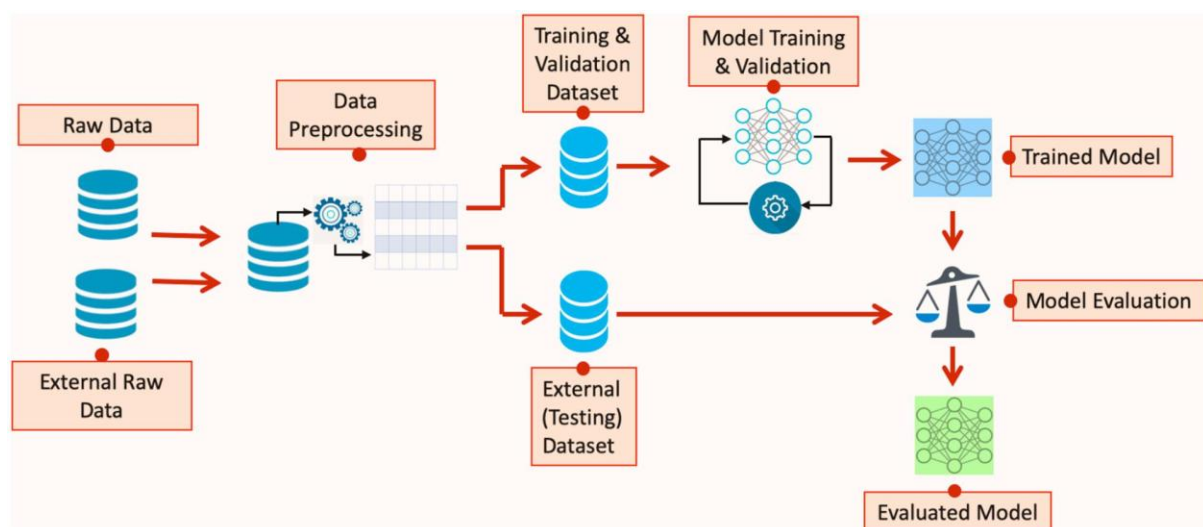
4.1 Overview of the Methodology

The proposed heart disease prediction system follows a systematic machine learning methodology. The process begins with collecting and understanding the Heart Disease Cleveland Dataset, followed by preprocessing the available patient information. The processed data is then used to train three different supervised classification algorithms: **Support Vector Machine (SVM)**, **Gaussian Naive Bayes**, and **Decision Tree**.

The main objective of applying multiple algorithms is to compare their predictive performance on the same dataset. Each algorithm uses a different classification principle, allowing the project to analyse which approach performs better for the given heart disease prediction problem.

The overall methodology can be represented as:

Dataset → Data Preprocessing → Feature Transformation → Train-Test Split → Model Training → Prediction → Model Evaluation → Comparison



4.2 Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning algorithm commonly used for classification problems. The main purpose of SVM is to identify a suitable decision boundary that separates observations belonging to different classes.

For a binary classification problem, SVM attempts to find a hyperplane that provides the maximum possible separation between the two classes. The observations closest to the decision boundary are known as **support vectors**. These observations play an important role in determining the position of the optimal separating boundary.

In this project, SVM is applied to classify patients into the **Disease** and **No Disease** categories. Since SVM can be sensitive to the scale of input features, feature scaling is included in the

preprocessing process. The processed patient attributes are supplied to the SVM model, which learns the classification boundary from the training data.

SVM is particularly useful when the relationship between the input features and classes can be effectively separated in the transformed feature space. The trained model is subsequently used to predict the classes of test samples and new patient records.

4.3 Gaussian Naive Bayes

Naive Bayes is a probabilistic supervised learning algorithm based on Bayes' theorem. It calculates the probability of a class based on the available input features.

The Gaussian Naive Bayes classifier assumes that the numerical features follow a Gaussian or normal distribution within each class. The algorithm calculates the probability of each possible class and selects the class with the highest posterior probability as the prediction.

One of the major advantages of Naive Bayes is its simplicity and relatively fast training process. It can perform well even when the dataset contains multiple features. However, the algorithm makes an assumption that the input features are conditionally independent given the class, which may not always hold for real-world healthcare data.

In this project, **GaussianNB** is trained using the processed heart disease dataset. The model produces predictions for the testing data and probability estimates that can be used for further evaluation, including ROC-AUC analysis.

4.4 Decision Tree

A **Decision Tree** is a supervised machine learning algorithm that performs classification by creating a tree-like structure of decisions. Each internal node represents a condition based on a feature, each branch represents an outcome of that condition, and the leaf nodes represent the final class prediction.

The Decision Tree algorithm selects features and creates decision rules that divide the training data into different groups. The process continues until suitable stopping conditions are reached. The resulting tree can be followed from the root node to a leaf node to determine the predicted class for a particular patient.

One advantage of Decision Trees is their interpretability. Unlike some complex models, a tree can be visually represented and its decision-making process can be understood through a sequence of rules.

In this project, the Decision Tree model is trained using the processed heart disease data. It predicts whether test patients belong to the Disease or No Disease class and is evaluated using the same metrics used for the other models.

4.5 Model Training Process

After completing data preprocessing, the training dataset is provided to each machine learning algorithm. The three models are trained independently using the same prepared input features and target variable.

For SVM, the model learns a decision boundary that separates the two target classes. Gaussian Naive Bayes learns the probability distributions associated with the input features and target classes. The Decision Tree learns a sequence of feature-based decision rules from the training observations.

Once training is completed, each model is applied to the testing dataset. The predicted results are compared with the actual target values to determine the performance of each classifier.

4.6 Model Prediction

The trained models generate predictions for observations that were not used during training. This is important because the main purpose of a predictive model is to generalize its learned patterns to new data.

For every test patient, the models produce a predicted classification. The predictions can then be compared with the actual classifications to determine whether the model correctly identified the patient's category.

The project also demonstrates prediction using new patient input. This provides an example of how the trained models can be used after development to classify previously unseen patient information.

4.7 Comparative Methodology

The three algorithms are evaluated using the same testing data and common performance measures. This makes it possible to compare their effectiveness objectively.

The project considers:

- **Accuracy** – overall percentage of correct predictions.
- **Precision** – correctness of positive predictions.
- **Recall** – ability to identify actual positive cases.
- **F1-Score** – balance between precision and recall.
- **ROC-AUC** – ability to distinguish between the two classes.
- **Confusion Matrix** – detailed representation of correct and incorrect predictions.
- **Classification Report** – combined summary of precision, recall, F1-score, and support.

The notebook specifically uses **Accuracy, Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix, and ROC Curve** for model evaluation.

This methodology provides a complete framework for developing the heart disease prediction system and forms the basis for the implementation and evaluation stages presented in the following sections.

5. Implementation

5.1 Development Environment

The heart disease prediction system is implemented using **Python** because it provides extensive libraries for data analysis, preprocessing, visualization, and machine learning. The implementation follows a complete machine learning workflow starting from dataset loading and data inspection and continuing through preprocessing, model training, prediction, and evaluation.

The major Python libraries used in the project are:

- **Pandas** – used for reading and manipulating the dataset.
- **NumPy** – used for numerical operations.
- **Matplotlib** – used for plotting graphs.
- **Seaborn** – used for statistical visualization and confusion matrices.
- **Scikit-learn** – used for preprocessing, classification algorithms, and evaluation metrics.

The notebook uses OneHotEncoder, StandardScaler, ColumnTransformer, and Pipeline for preprocessing. The classification algorithms implemented are SVC, GaussianNB, and DecisionTreeClassifier.

5.2 Importing Required Libraries

The first implementation step is importing the libraries required for data processing, visualization, model training, and evaluation.

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.svm import SVC
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix,
    roc_curve
)
```

These libraries provide all the required functionality for implementing the proposed heart disease classification system.

5.3 Loading the Dataset

The Heart Disease Cleveland Dataset is loaded using the Pandas library. The dataset is stored in a DataFrame, which makes it convenient to inspect and manipulate the records.

```
df = pd.read_csv("heart_disease.csv")
```

```
print("Dataset Shape:", df.shape)
```

```
display(df.head())
```

The head() function displays the first few records of the dataset. This allows the structure and values of the different attributes to be examined before preprocessing.

The dataset contains patient-related attributes that are used as input features, along with a target variable representing the heart disease classification.

5.4 Initial Data Inspection

After loading the dataset, its structure and data types are examined.

```
print("Dataset Information:")
```

```
print(df.info())
```

```
print("\nDataset Description:")
```

```
display(df.describe())
```

```
print("\nFirst Five Records:")
```

```
display(df.head())
```

The inspection helps determine:

- Number of records.
- Number of features.
- Data types.
- Numerical and categorical attributes.
- Statistical characteristics of numerical variables.
- Structure of the target variable.

This step helps identify the preprocessing operations required before model training.

5.5 Checking Missing Values

Missing values are checked before training the models.

```
print("Missing Values in Each Column:")
```

```
print(df.isnull().sum())
```

```
print("\nTotal Missing Values:",
```

```
df.isnull().sum().sum())
```

Checking missing values is important because incomplete observations can affect the training process. If missing values are present, suitable imputation techniques can be applied before proceeding.

5.6 Exploratory Data Analysis

Exploratory Data Analysis is performed to understand the distribution of the target variable and identify useful patterns in the dataset.

The target variable is visualized using a count plot.

```
plt.figure(figsize=(7, 5))

ax = sns.countplot(x="target", data=df)

plt.title("Heart Disease Class Distribution")
plt.xlabel("Target")
plt.ylabel("Number of Patients")
plt.xticks(
    [0, 1],
    ["No Disease", "Disease"]
)

for container in ax.containers:
    ax.bar_label(container)

plt.show()
```



This graph displays the number of observations belonging to the two target classes. It helps determine how the patient records are distributed between the **No Disease** and **Disease** categories.

5.7 Separating Features and Target

The dataset is divided into input features and the target variable.

```
X = df.drop("target", axis=1)
```

```
y = df["target"]
```

```
print("Feature Shape:", X.shape)
```

```
print("Target Shape:", y.shape)
```

Here, X contains the patient-related attributes that will be supplied to the machine learning algorithms. The variable y contains the corresponding target class.

The target is not included among the input features because it represents the output that the model is expected to predict.

5.8 Identifying Numerical and Categorical Features

The input features are divided into numerical and categorical features.

```
categorical_features = X.select_dtypes(  
    include=["object", "category"]  
).columns
```

```
numerical_features = X.select_dtypes(  
    exclude=["object", "category"]  
).columns
```

```
print("Categorical Features:")  
print(list(categorical_features))  
print("\nNumerical Features:")  
print(list(numerical_features))
```

This separation is necessary because numerical and categorical features require different preprocessing techniques.

Numerical features are scaled using `StandardScaler`, while categorical features are converted into numerical representations using `OneHotEncoder`.

5.9 Creating the Preprocessing Pipeline

A preprocessing pipeline is created using `ColumnTransformer`.

```
preprocessor = ColumnTransformer(  
    transformers=[  
        (  
            "num",  
            StandardScaler(),  
            numerical_features  
        ),  
        (  
            "cat",  
            OneHotEncoder(  
                handle_unknown="ignore"  
            ),  
            categorical_features  
        )  
    ]  
)
```

The preprocessing pipeline performs two major operations:

1. **StandardScaler** standardizes numerical features.
2. **OneHotEncoder** converts categorical features into numerical form.

Using a preprocessing pipeline ensures that the same transformations are consistently applied to training and testing data.

5.10 Splitting the Dataset

The dataset is divided into training and testing subsets.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)
```

```
print("Training Data:", X_train.shape)
```

```
print("Testing Data:", X_test.shape)
```

The training dataset is used to train the models, while the testing dataset is used to evaluate their performance.

The stratify=y parameter helps maintain the class distribution of the target variable in both training and testing datasets.

5.11 Support Vector Machine Implementation

Support Vector Machine is the first classification algorithm implemented in the project.

The SVM model attempts to find an effective decision boundary that separates the **Disease** and **No Disease** classes.

```
svm_model = Pipeline([  
    (  
        "preprocessing",  
        preprocessor  
    ),  
    (  
        "classifier",  
        SVC(  
            probability=True,
```

```

        random_state=42
    )
)
])

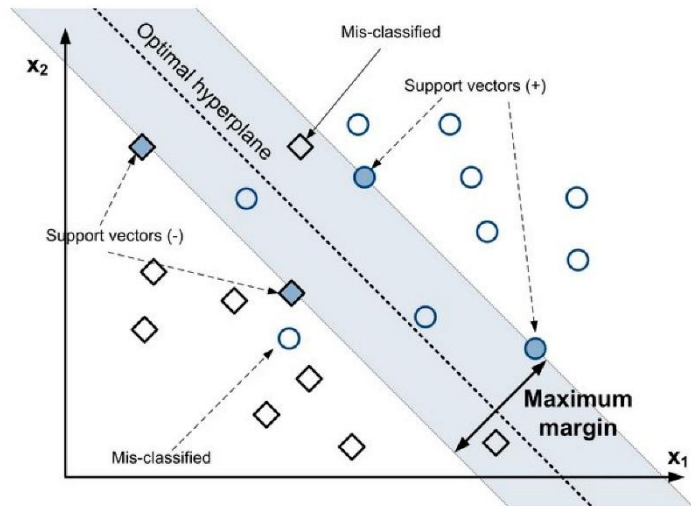
svm_model.fit(X_train, y_train)

y_pred_svm =
svm_model.predict(X_test)

y_prob_svm =
svm_model.predict_proba(X_test)[:, 1]

print("SVM training completed.")

```



The preprocessing stage is integrated with the SVM classifier through a pipeline. The trained model is then used to generate predictions for the testing dataset.

The `probability=True` parameter enables probability estimates, which are required for calculating the ROC-AUC score.

SVM Workflow

Input Data → Preprocessing → Feature Scaling/Encoding → SVM Training → Prediction → Evaluation

5.12 Naive Bayes Implementation

The second algorithm implemented is **Gaussian Naive Bayes**.

Gaussian Naive Bayes is a probability-based classification algorithm. It estimates the probability of each class based on the input features.

```
# Fit preprocessing on training data
```

```
X_train_processed = preprocessor.fit_transform(X_train)
```

```
# Transform testing data
```

```
X_test_processed = preprocessor.transform(X_test)
```

```

# Convert sparse matrix to dense if required
if hasattr(X_train_processed, "toarray"):
    X_train_processed = X_train_processed.toarray()

if hasattr(X_test_processed, "toarray"):
    X_test_processed = X_test_processed.toarray()

# Create Naive Bayes model
nb_model = GaussianNB()

# Train model
nb_model.fit(
    X_train_processed,
    y_train
)

# Prediction
y_pred_nb = nb_model.predict(
    X_test_processed
)

# Prediction probabilities
y_prob_nb = nb_model.predict_proba(
    X_test_processed
)[: , 1]

print("Naive Bayes training completed.")

```

The preprocessing transformation is performed before applying Gaussian Naive Bayes. The model then predicts the class of each testing observation.

The probability estimates generated by the model are also used for ROC-AUC evaluation.

Naive Bayes Workflow

Input Data → Preprocessing → Probability Estimation → Naive Bayes Training → Prediction → Evaluation

5.13 Decision Tree Implementation

The third algorithm is the **Decision Tree Classifier**.

Decision Tree creates a tree-like structure consisting of decision nodes, branches, and leaf nodes. Each decision is based on one or more input features.

```
dt_model = Pipeline([
    (
        "preprocessing",
        preprocessor
    ),
    (
        "classifier",
        DecisionTreeClassifier(
            random_state=42
        )
    )
])
```

```
dt_model.fit(
    X_train,
    y_train
)
```

```
y_pred_dt = dt_model.predict(
    X_test
)
```



```
y_prob_dt = dt_model.predict_proba(  
    X_test  
)[: , 1]
```

```
print("Decision Tree training completed.")
```

The model learns decision rules from the training data and applies those rules to classify the testing observations.

Decision Tree Workflow

Input Data → Preprocessing → Decision Rules → Tree Construction → Prediction → Evaluation

5.14 Model Evaluation Function

A common evaluation function is created so that all three algorithms can be evaluated using the same metrics.

```
def evaluate_model(  
    name,  
    y_true,  
    y_pred,  
    y_prob  
):  
  
    accuracy = accuracy_score(  
        y_true,  
        y_pred  
    )  
  
    precision = precision_score(  
        y_true,  
        y_pred,
```

```

        zero_division=0
    )

recall = recall_score(
    y_true,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_true,
    y_pred,
    zero_division=0
)

auc = roc_auc_score(
    y_true,
    y_prob
)

print("\n", name)
print("-" * 40)

print(
    "Accuracy :",
    round(accuracy, 4)
)

print(

```

```
"Precision:",  
    round(precision, 4)  
)  
  
print(  
    "Recall :",  
    round(recall, 4)  
)  
  
print(  
    "F1-Score :",  
    round(f1, 4)  
)  
  
print(  
    "ROC-AUC :",  
    round(auc, 4)  
)  
  
print("\nClassification Report:")  
  
print(  
    classification_report(  
        y_true,  
        y_pred,  
        target_names=[  
            "No Disease",  
            "Disease"  
        ]  
    )  
)
```

```

    )

    )

    return [
        accuracy,
        precision,
        recall,
        f1,
        auc
    ]

```

This function calculates five important performance metrics and generates the classification report for each model.

5.15 Evaluating SVM

```

svm_metrics = evaluate_model(
    "SVM",
    y_test,
    y_pred_svm,
    y_prob_svm
)

```

The SVM model obtained approximately:

Metric	SVM
Accuracy	88.52%
Precision	83.87%
Recall	92.86%
F1-Score	88.14%
ROC-AUC	96.43%

The results indicate strong performance, particularly in terms of recall and ROC-AUC.

5.16 Evaluating Naive Bayes

```
nb_metrics = evaluate_model(  
    "Naive Bayes",  
    y_test,  
    y_pred_nb,  
    y_prob_nb  
)
```

The Naive Bayes model obtained:

Metric	Naive Bayes
Accuracy	72.13%
Precision	64.10%
Recall	89.29%
F1-Score	74.63%
ROC-AUC	82.68%

The model provides relatively high recall but lower accuracy and precision compared with SVM.

5.17 Evaluating Decision Tree

```
dt_metrics = evaluate_model(  
    "Decision Tree",  
    y_test,  
    y_pred_dt,  
    y_prob_dt  
)
```

The Decision Tree model obtained:

Metric	Decision Tree
Accuracy	73.77%
Precision	70.00%
Recall	75.00%

F1-Score	72.41%
ROC-AUC	78.90%

The Decision Tree provides moderate classification performance across the evaluated metrics.

5.18 Confusion Matrix for SVM

A confusion matrix is generated to examine the correct and incorrect predictions made by the SVM model.

```
cm_svm = confusion_matrix(
    y_test,
    y_pred_svm
)
```

```
plt.figure(figsize=(6, 5))
```

```
sns.heatmap(
    cm_svm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=[
        "No Disease",
        "Disease"
    ],
    yticklabels=[
        "No Disease",
        "Disease"
    ]
)
```

```
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.show()
```

The matrix contains four categories:

- True Negative
- False Positive
- False Negative
- True Positive

The confusion matrix allows the classification errors of the model to be examined in detail.

5.19 Confusion Matrices for All Models

```
models = {
    "SVM": y_pred_svm,
    "Naive Bayes": y_pred_nb,
    "Decision Tree": y_pred_dt
}
```

```
for name, predictions in models.items():
```

```
    cm = confusion_matrix(
        y_test,
        predictions
    )
```

```
    plt.figure(figsize=(6, 5))
```

```
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
```

```

cmap="Blues",
xticklabels=[
    "No Disease",
    "Disease"
],
yticklabels=[
    "No Disease",
    "Disease"
]
)
plt.title(
    name + " Confusion Matrix"
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

Generating confusion matrices for all three models provides a direct visual comparison of their classification errors.

5.20 ROC Curve Implementation

ROC curves are used to compare the ability of the models to distinguish between the two target classes.

```

fpr_svm, tpr_svm, _ = roc_curve(
    y_test,
    y_prob_svm
)

```

```

fpr_nb, tpr_nb, _ = roc_curve(
    y_test,
    y_prob_nb
)

```



```

)
fpr_dt, tpr_dt, _ = roc_curve(
    y_test,
    y_prob_dt
)
plt.figure(figsize=(8, 6))
plt.plot(
    fpr_svm,
    tpr_svm,
    label="SVM"
)
plt.plot(
    fpr_nb,
    tpr_nb,
    label="Naive Bayes"
)
plt.plot(
    fpr_dt,
    tpr_dt,
    label="Decision Tree"
)
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--"
)
plt.xlabel(
    "False Positive Rate"
)

```

```
plt.ylabel(  
    "True Positive Rate"  
)  
plt.title(  
    "ROC Curve Comparison"  
)  
plt.legend()  
plt.show()
```

The ROC curve provides a graphical representation of the trade-off between the true-positive rate and false-positive rate.

5.21 Model Performance Comparison

The final performance comparison is presented using a DataFrame.

```
results = pd.DataFrame(  
    [  
        svm_metrics,  
        nb_metrics,  
        dt_metrics  
    ],  
    columns=[  
        "Accuracy",  
        "Precision",  
        "Recall",  
        "F1-Score",  
        "ROC-AUC"  
    ],  
    index=[  
        "SVM",  
        "Naive Bayes",
```

```

        "Decision Tree"
    ]
)

```

```

display(
    results.round(4)
)

```

The resulting comparison is:

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
SVM	88.52%	83.87%	92.86%	88.14%	96.43%
Naive Bayes	72.13%	64.10%	89.29%	74.63%	82.68%
Decision Tree	73.77%	70.00%	75.00%	72.41%	78.90%

5.22 Performance Comparison Graph

```

results_percentage = results * 100
results_percentage.plot(
    kind="bar",
    figsize=(10, 6)
)
plt.title(
    "Performance Comparison of Machine Learning Models"
)
plt.xlabel("Machine Learning Model")
plt.ylabel("Performance (%)")
plt.xticks(rotation=0)
plt.legend()
plt.grid(axis="y")

plt.show()

```

5.23 Implementation Summary

The implementation successfully establishes a complete machine learning pipeline for heart disease prediction. The process begins with importing the required Python libraries and loading the dataset. The data is then inspected, missing values are checked, features are separated from the target, and categorical and numerical attributes are identified.

The preprocessing stage uses **One-Hot Encoding and Standard Scaling** to prepare the input data. The dataset is divided into training and testing subsets, after which three classification models—**SVM, Gaussian Naive Bayes, and Decision Tree**—are developed.

Each model generates predictions on the testing dataset. The models are evaluated using accuracy, precision, recall, F1-score, and ROC-AUC, while confusion matrices and ROC curves provide graphical analysis. The comparison shows that **SVM provides the strongest overall performance**, achieving 88.52% accuracy and 96.43% ROC-AUC in the project results.

The implementation also demonstrates how trained models can be used to predict the class of new patient data. Therefore, this section covers the complete technical implementation required for the proposed heart disease prediction system.

Performance comparison of three models

Comparison of accuracy, precision, recall, F1-score, and ROC-AUC for the three implemented classifiers.

model	accuracy	precision	recall	f1	auc
SVM	88.52	83.87	92.86	88.14	96.43
Naive Bayes	72.13	64.1	89.29	74.63	82.68
Decision Tree	73.77	70	75	72.41	78.9

6. Model Evaluation

6.1 Introduction

Model evaluation is an important stage of the heart disease prediction project. After training the **Support Vector Machine (SVM)**, **Gaussian Naive Bayes**, and **Decision Tree** models, their performance is evaluated using the testing dataset. The purpose of evaluation is to determine how accurately each model can classify patients into the **No Disease** and **Disease** categories.

In this project, multiple evaluation metrics are used instead of depending only on accuracy. The selected metrics are **Accuracy**, **Precision**, **Recall**, **F1-Score**, and **ROC-AUC**. In addition, **classification reports**, **confusion matrices**, and **ROC curves** are used to provide a detailed understanding of the model predictions.

6.2 Accuracy

Accuracy measures the percentage of total predictions that are correctly classified.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where **TP** represents True Positive, **TN** represents True Negative, **FP** represents False Positive, and **FN** represents False Negative.

The accuracy obtained by the three models is:

Model	Accuracy
SVM	88.52%
Naive Bayes	72.13%
Decision Tree	73.77%

SVM achieved the highest accuracy of **88.52%**, indicating that it correctly classified a larger proportion of the testing observations than Naive Bayes and Decision Tree.

6.3 Precision

Precision measures how accurately the model identifies positive predictions.

$$Precision = \frac{TP}{TP + FP}$$

A high precision value means that a greater proportion of observations predicted as disease actually belong to the disease class.

The precision values are:

Model	Precision
SVM	83.87%
Naive Bayes	64.10%
Decision Tree	70.00%

SVM achieved the highest precision. This indicates that its positive predictions were more accurate compared with the predictions produced by Naive Bayes and Decision Tree.

6.4 Recall

Recall measures the ability of a model to correctly identify actual positive cases.

$$Recall = \frac{TP}{TP + FN}$$

For a heart disease prediction system, recall is an important metric because identifying actual disease cases is an important part of the classification task.

The obtained recall values are:

Model	Recall
SVM	92.86%
Naive Bayes	89.29%
Decision Tree	75.00%

SVM achieved the highest recall of **92.86%**, followed by Naive Bayes with **89.29%**. Decision Tree achieved a recall of **75.00%**.

6.5 F1-Score

F1-score provides a balance between precision and recall. It is particularly useful when both types of classification errors need to be considered.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

The F1-score values obtained are:

Model	F1-Score
SVM	88.14%
Naive Bayes	74.63%

Decision Tree	72.41%
---------------	--------

SVM achieved the highest F1-score, indicating a better balance between precision and recall.

6.6 ROC-AUC

The **Receiver Operating Characteristic (ROC)** curve is used to analyse the classification ability of a model across different classification thresholds. The **Area Under the Curve (AUC)** provides a numerical representation of the model's ability to distinguish between the two classes.

The ROC-AUC results are:

Model	ROC-AUC
SVM	96.43%
Naive Bayes	82.68%
Decision Tree	78.90%

SVM achieved the highest ROC-AUC value of **96.43%**, demonstrating the strongest class-separation performance among the three models.

6.7 Classification Report

A classification report provides detailed performance information for each target class. It includes **precision, recall, F1-score, and support** for the No Disease and Disease classes.

For the Naive Bayes model, the Disease class achieved a precision of **0.64** and recall of **0.89**, while the No Disease class achieved a precision of **0.86** and recall of **0.58**.

For the Decision Tree model, the Disease class achieved a precision of **0.70** and recall of **0.75**. The No Disease class achieved a precision of **0.77** and recall of **0.73**.

These class-specific values show that overall accuracy alone does not provide a complete picture of model behaviour.

6.8 Confusion Matrix

A confusion matrix provides a detailed representation of the predicted and actual classes. It contains four categories:

- **True Positive (TP):** Disease correctly predicted as Disease.
- **True Negative (TN):** No Disease correctly predicted as No Disease.
- **False Positive (FP):** No Disease incorrectly predicted as Disease.

- **False Negative (FN):** Disease incorrectly predicted as No Disease.

The confusion matrix is particularly useful for understanding the types of errors made by each model.

6.9 ROC Curve Comparison

The ROC curve is generated for all three models to visually compare their classification performance.

```
fpr_svm, tpr_svm, _ = roc_curve(
    y_test,
    y_prob_svm
)
```

```
fpr_nb, tpr_nb, _ = roc_curve(
    y_test,
    y_prob_nb
)
```

```
fpr_dt, tpr_dt, _ = roc_curve(
    y_test,
    y_prob_dt
)
```

```
plt.figure(figsize=(8, 6))
```

```
plt.plot(
    fpr_svm,
    tpr_svm,
    label="SVM"
)
```

```
plt.plot(
```



```
fpr_nb,  
tpr_nb,  
label="Naive Bayes"  
)
```

```
plt.plot(  
    fpr_dt,  
    tpr_dt,  
    label="Decision Tree"  
)
```

```
plt.plot(  
    [0, 1],  
    [0, 1],  
    linestyle="--"  
)
```

```
plt.xlabel("False Positive Rate")  
plt.ylabel("True Positive Rate")
```

```
plt.title(  
    "ROC Curve Comparison"  
)
```

```
plt.legend()  
plt.show()
```

The model with the ROC curve closer to the upper-left region generally demonstrates stronger classification performance. Based on the ROC-AUC values obtained in this project, SVM performs better than Naive Bayes and Decision Tree.

6.10 Overall Performance Comparison

The complete evaluation results are summarized below.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
SVM	88.52%	83.87%	92.86%	88.14%	96.43%
Naive Bayes	72.13%	64.10%	89.29%	74.63%	82.68%
Decision Tree	73.77%	70.00%	75.00%	72.41%	78.90%

The comparison clearly shows that **SVM achieved the highest performance across all five evaluation metrics**. Naive Bayes achieved a relatively high recall but lower precision and accuracy. Decision Tree produced moderate results across the evaluated metrics.

Therefore, based on the experimental results, **SVM is the best-performing model among the three algorithms used in this project**. Its high accuracy, recall, F1-score, and ROC-AUC indicate that it provides the strongest overall classification performance for the given heart disease dataset.

Accuracy comparison

model	accuracy
SVM	88.52
Naive Bayes	72.13
Decision Tree	73.77

Recall comparison

model	recall
SVM	92.86
Naive Bayes	89.29
Decision Tree	75

ROC-AUC comparison

model	auc
SVM	96.43
Naive Bayes	82.68
Decision Tree	78.9

7. Results and Analysis

7.1 Introduction

After completing the preprocessing and training stages, the three machine learning models—**Support Vector Machine (SVM)**, **Naive Bayes**, and **Decision Tree**—are evaluated using the testing data. The purpose of this section is to analyse the obtained results and compare the models using different performance measures.

The evaluation considers **accuracy, precision, recall, F1-score, and ROC-AUC**. Confusion matrices and ROC curves are also used to understand the classification behaviour of the models in greater detail.

7.2 SVM Results

The SVM model produced the strongest results among the three algorithms used in the project.

Metric	SVM
Accuracy	88.52%
Precision	83.87%
Recall	92.86%
F1-Score	88.14%
ROC-AUC	96.43%

The accuracy of **88.52%** indicates that SVM correctly classified most of the observations in the testing dataset. Its precision of **83.87%** indicates that most of the patients predicted as belonging to the Disease class were correctly classified.

The model achieved a recall of **92.86%**, which was the highest recall among the three algorithms. This shows that SVM was particularly effective at identifying observations belonging to the positive disease class.

The F1-score of **88.14%** demonstrates a strong balance between precision and recall. Furthermore, the ROC-AUC value of **96.43%** indicates strong class-separation performance.

Overall, SVM provided the best results in this experiment.

7.3 Naive Bayes Results

The Gaussian Naive Bayes model achieved moderate performance.

Metric	Naive Bayes
Accuracy	72.13%
Precision	64.10%

Recall	89.29%
F1-Score	74.63%
ROC-AUC	82.68%

The model achieved an accuracy of **72.13%**. Although its overall accuracy was lower than SVM, Naive Bayes achieved a relatively high recall of **89.29%**.

The precision of **64.10%** was considerably lower than the SVM precision. This indicates that although the model was able to identify many positive cases, a larger proportion of its positive predictions were incorrect compared with SVM.

The F1-score of **74.63%** indicates a moderate balance between precision and recall. Its ROC-AUC of **82.68%** was also lower than that of SVM.

7.4 Decision Tree Results

The Decision Tree model produced the following results:

Metric	Decision Tree
Accuracy	73.77%
Precision	70.00%
Recall	75.00%
F1-Score	72.41%
ROC-AUC	78.90%

The Decision Tree achieved an accuracy of **73.77%**, which was slightly higher than Naive Bayes. Its precision was **70.00%**, while its recall was **75.00%**.

The F1-score of **72.41%** shows moderate classification performance. The ROC-AUC value of **78.90%** was the lowest among the three models, indicating comparatively weaker class-separation performance.

Although Decision Tree provides an interpretable tree-based structure, its predictive performance in this experiment was lower than that of SVM.

7.5 Comparative Analysis

The performance of all three models is summarized below.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
SVM	88.52%	83.87%	92.86%	88.14%	96.43%
Naive Bayes	72.13%	64.10%	89.29%	74.63%	82.68%
Decision Tree	73.77%	70.00%	75.00%	72.41%	78.90%

The comparison demonstrates that **SVM outperformed both Naive Bayes and Decision Tree across all major evaluation metrics**. Its accuracy, precision, recall, F1-score, and ROC-AUC were all higher.

Naive Bayes performed better than Decision Tree in terms of recall, F1-score, and ROC-AUC, whereas Decision Tree achieved slightly better accuracy and precision than Naive Bayes.

7.6 Accuracy Analysis

SVM achieved an accuracy of **88.52%**, which is substantially higher than Naive Bayes at **72.13%** and Decision Tree at **73.77%**.

This indicates that SVM was able to learn a more effective classification boundary for the given dataset. The difference in accuracy also demonstrates that selecting an appropriate machine learning algorithm can have a significant effect on prediction performance.

7.7 Precision Analysis

SVM obtained the highest precision at **83.87%**. Decision Tree achieved **70.00%**, while Naive Bayes obtained **64.10%**.

The higher precision of SVM indicates that it generated fewer incorrect positive classifications compared with the other two algorithms.

7.8 Recall Analysis

SVM achieved the highest recall at **92.86%**, followed by Naive Bayes at **89.29%** and Decision Tree at **75.00%**.

The high recall values of SVM and Naive Bayes show that both models were effective at identifying positive disease cases. However, SVM provided the best balance because it combined high recall with substantially higher precision.

7.9 F1-Score Analysis

The F1-score results further demonstrate the advantage of SVM.

- **SVM:** 88.14%
- **Naive Bayes:** 74.63%
- **Decision Tree:** 72.41%

Since the F1-score combines precision and recall, SVM's higher value indicates that it provided the most balanced classification performance.

7.10 ROC-AUC Analysis

SVM achieved a ROC-AUC of **96.43%**, which was considerably higher than Naive Bayes (**82.68%**) and Decision Tree (**78.90%**).

This indicates that SVM demonstrated the strongest ability to distinguish between the Disease and No Disease classes in the testing data.

7.11 Overall Findings

The experimental results provide the following major observations:

1. **SVM achieved the highest overall performance.**
2. SVM obtained the highest accuracy of **88.52%**.
3. SVM achieved the highest recall of **92.86%**.
4. SVM obtained the highest F1-score of **88.14%**.
5. SVM achieved the highest ROC-AUC of **96.43%**.
6. Naive Bayes achieved a relatively high recall of **89.29%** but had lower precision.
7. Decision Tree achieved slightly higher accuracy and precision than Naive Bayes but had lower recall and ROC-AUC.
8. The results demonstrate the importance of comparing multiple machine learning algorithms rather than selecting a model based on a single metric.

Based on the complete experimental comparison, **SVM is the most suitable model among the three algorithms for the implemented heart disease prediction task.** Its consistent performance across all evaluation measures makes it the strongest model in this study.

Overall model performance

Comparison of the five evaluation metrics across SVM, Naive Bayes, and Decision Tree.

model	accuracy	precision	recall	f1	auc
SVM	88.52	83.87	92.86	88.14	96.43
Naive Bayes	72.13	64.1	89.29	74.63	82.68
Decision Tree	73.77	70	75	72.41	78.9

8. Prediction and Discussion

8.1 Prediction on New Patient Data

After training the three machine learning models, they can be used to predict the outcome for new patient data. The new patient information is first processed using the same preprocessing steps applied during training. The trained models then classify the patient into either **Disease** or **No Disease**.

```
new_patient = pd.DataFrame({  
    # Enter the same features used during training  
})
```

```
svm_prediction = svm_model.predict(new_patient)
```

```
dt_prediction = dt_model.predict(new_patient)
```

```
print("SVM:", "Disease" if svm_prediction[0] == 1 else "No Disease")
```

```
print("Decision Tree:", "Disease" if dt_prediction[0] == 1 else "No Disease")
```

For Naive Bayes, the new data is transformed using the preprocessing object before prediction.

8.2 Discussion of Results

The experimental results show that **SVM performed better than Naive Bayes and Decision Tree**. SVM achieved **88.52% accuracy**, **83.87% precision**, **92.86% recall**, **88.14% F1-score**, and **96.43% ROC-AUC**.

Naive Bayes achieved **72.13% accuracy** and a relatively high recall of **89.29%**, but its precision was lower at **64.10%**. Decision Tree achieved **73.77% accuracy**, with **70.00% precision** and **75.00% recall**.

Overall, SVM provided the best balance between precision and recall and showed the strongest ability to distinguish between the two classes.

8.3 Advantages

The proposed system provides several advantages:

- Compares three different machine learning algorithms.
- Uses appropriate data preprocessing.

- Evaluates models using multiple performance metrics.
- Provides confusion matrices and ROC curves.
- Supports prediction on new patient data.
- Helps identify the best-performing classification model.

8.4 Limitations

The model performance depends on the dataset used for training and testing. Results may differ when the models are applied to other datasets or patient populations. The system also provides machine learning-based predictions and should not be considered a replacement for professional medical diagnosis.

8.5 Summary

The prediction stage demonstrates the practical use of the trained models for classifying new patient records. Based on the experimental results, **SVM is the best-performing model** among the three algorithms used in this project. The results demonstrate the potential of machine learning for heart disease classification while highlighting the importance of proper preprocessing and model evaluation.

9. Conclusion

This project successfully developed a machine learning-based system for **heart disease prediction** using three classification algorithms: **Support Vector Machine (SVM), Naive Bayes, and Decision Tree**. The main objective was to analyse patient-related attributes and determine whether a patient belongs to the Disease or No Disease category.

The dataset was processed using appropriate **data preprocessing techniques**, including data inspection, feature selection, categorical feature encoding, feature scaling, and train-test splitting. These preprocessing steps helped prepare the dataset in a suitable format for applying the selected machine learning algorithms.

The performance of the models was evaluated using **accuracy, precision, recall, F1-score, and ROC-AUC**. Confusion matrices, classification reports, and ROC curves were also used to provide a detailed analysis of the predictions. This comprehensive evaluation helped in understanding the strengths and weaknesses of each model.

Among the three algorithms, **SVM achieved the best overall performance**, with **88.52% accuracy, 83.87% precision, 92.86% recall, 88.14% F1-score, and 96.43% ROC-AUC**. Naive Bayes and Decision Tree achieved comparatively lower overall performance, although Naive Bayes showed a relatively high recall.

Overall, the project demonstrates that machine learning can be effectively applied to heart disease classification using structured patient data. The comparative analysis shows that **SVM was the most effective model for this particular dataset and experiment**. The developed system provides a useful academic foundation for healthcare prediction applications, while its results should be considered supportive analysis and not a replacement for professional medical diagnosis.

10. References

- [1] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, **“Heart Disease,” UCI Machine Learning Repository**, 1989, doi: 10.24432/C52P4X. **Dataset:** UCI Heart Disease Dataset.
- [2] R. Detrano et al., “International application of a new probability algorithm for the diagnosis of coronary artery disease,” *American Journal of Cardiology*, 1989.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995.
- [5] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997.
- [6] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [7] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [8] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA, USA: Morgan Kaufmann, 2011.
- [9] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.
- [10] I. Rish, “An empirical study of the naive Bayes classifier,” *IBM Research Report*, RC 22230, 2001.
- [11] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [12] J. Davis and M. Goadrich, “The relationship between Precision-Recall and ROC curves,” in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 233–240.
- [13] Scikit-learn Developers, **“Support Vector Machines,” Scikit-learn Documentation**. SVM Documentation
- [14] Scikit-learn Developers, **“Naive Bayes,” Scikit-learn Documentation**. Naive Bayes Documentation
- [15] Scikit-learn Developers, **“Decision Trees,” Scikit-learn Documentation**. Decision Tree Documentation